Noise-Aware Promotion for Self-Evolving GPU Kernels

A statistical extension to cross-run kernel optimization

Lee Si Yuan

Independent Researcher

siyuanlee.cs@gmail.com

November 5, 2026

Abstract

Self-evolving kernel optimizers preserve validated kernels as references for later runs. A promotion decision based on a small number of timing measurements can preserve a benchmark outlier instead of a true improvement. We propose a noise-aware promotion gate based on paired log-latency measurements and a minimum practical speedup. A candidate is promoted only when a one-sided confidence bound supports at least the configured gain. To examine the gate, we run a reproducible CPU Monte Carlo simulation with shared timing drift and independent jitter. At 32 paired repeats and a 1% minimum gain, the gate promotes a true 5% speedup in all 30,000 trials and a zero-speedup candidate in 0.01% of trials. An otherwise identical unpaired gate promotes the 5% candidate in 99.85% of trials, illustrating the value of pairing under the simulation's shared-drift assumption. This is a methodological simulation, not a GPU benchmark; its results motivate validating the policy with real, randomized device measurements.

1. Introduction

ForgeKernel describes a self-evolving kernel optimization process that retains an accepted implementation as the next reference and carries validated hardware knowledge across runs [1]. Such a process needs a reliable rule for deciding when an implementation is sufficiently faster to replace its reference. Kernel timings fluctuate with device state, clock behavior, contention, and measurement overhead. Selecting the fastest observed run can therefore turn noise into persistent state.

This paper adds a small statistical component to that process. It asks whether pairing each

candidate timing with a nearby reference timing can reduce the uncertainty of a promotion decision when both measurements experience common environmental drift. We model log latency because differences in log latency correspond to multiplicative speedups. The study tests a fixed-sample promotion rule under explicit synthetic assumptions.

Our contribution is a practical promotion rule and a reproducible simulation of its false-promotion and detection rates. We do not reproduce ForgeKernel, use its source code, evaluate a GPU kernel, or claim an improvement to its measured performance.

2. Promotion rule

For pair i, let b_i and c_i be the logarithms of reference and candidate latency measured in the same experimental block. Define $d_i = c_i - b_i$. Negative values favor the candidate. Let $\bar{d}$ and s_d denote the sample mean and standard deviation of the paired differences, and let n be the number of pairs. For a configured minimum practical speedup g, promote only when

$$\bar{d} + z_{0.95}\, s_d / \sqrt{n} < -\log(1 + g),$$

where $z_{0.95} = 1.64485$. This requires the upper one-sided 95% normal bound on mean log latency difference to beat the practical-gain threshold. A fixed sample count is used for each decision; the rule does not permit repeated interim looks. A production system should use a calibrated critical value, control comparisons when screening many candidates, and preserve the prior reference if the gate does not pass.

The paired design assumes reference and candidate measurements share some transient conditions. Pairing can remove that common component from d_i; it cannot remove candidate-specific noise, systematic ordering effects, or workload changes. Measurement order should therefore be randomized or counterbalanced on hardware.

3. Experimental method

We implement the experiment in `noise_aware_promotion_experiment.py`, using only Python's standard library. The fixed seed is `20261105`; each point uses 30,000 Monte Carlo trials. In each pair, the simulated reference and candidate log latencies share a Gaussian drift with standard deviation 0.05. Each also receives independent Gaussian jitter with standard deviation 0.02. The candidate's mean log latency is shifted to represent true speedups of 0%, 0.5%, 1%, 2%,

or 5%.

We compare the paired gate above with an unpaired gate that uses the difference in sample means and the estimated standard error from two independent timing series. Both gates use the same one-sided normal critical value, fixed sample sizes of 8, 16, or 32, and a 1% minimum practical gain. The unpaired comparison is intentionally exposed to independent common-drift draws between reference and candidate series. This makes it a test of pairing under the stated model, not a claim that all real measurement protocols behave this way.

The full configuration and per-cell estimates are recorded in `results.json`. The Monte Carlo standard error near a 5% event rate is about 0.13 percentage points; near zero observed events, the resolution is limited by 30,000 trials.

4. Results

Promotion probability by true speedup and number of timing pairs:

Repeats	True speedup	Paired gate	Unpaired gate
8	0%	0.927%	0.003%
8	2%	29.253%	1.010%
8	5%	98.193%	39.923%
16	0%	0.157%	0.000%
16	2%	41.617%	0.787%
16	5%	99.987%	82.023%
32	0%	0.010%	0.000%
32	2%	62.470%	1.507%
32	5%	100.000%	99.853%

At zero speedup, the paired false-promotion rate falls as sample count increases. At 32 repeats it is 0.01% in this finite simulation (3 of 30,000 trials). At the same repeat count, the gate accepts 62.47% of candidates with a true 2% speedup and all 30,000 candidates with a true 5% speedup. The nonzero detection rate at exactly the 1% practical threshold is about 5%, as expected for a one-sided bound whose null boundary is that threshold.

In the zero-speedup trials, the average sample standard deviation of paired log differences is about 0.028, compared with about 0.053 for either unpaired timing series. The paired gate is substantially more sensitive in this model because it cancels shared drift. Its performance depends on the shared-drift assumption and should not be generalized to a particular GPU or benchmark suite without measurement.

5. Implications for kernel optimization

The promotion gate can be placed after functional correctness and before replacing the retained reference. It gives a self-evolving optimizer an explicit evidence requirement: a candidate must clear both timing uncertainty and a practical improvement floor. This can reduce persistence of noise-driven wins. The fixed-sample formulation also avoids the inflated false-positive rates that can result from repeatedly checking an unadjusted confidence bound.

For a device implementation, pairs should be interleaved and randomized, warm-up and synchronization policy should be fixed, and timings should be collected under controlled workload conditions. Multiple candidates require a family-wise or sequential-testing policy. Correctness, numerical behavior, memory safety, and regressions on other input configurations remain separate promotion requirements.

6. Limitations

This study uses synthetic Gaussian log-latency data and a normal critical value. It contains no measured GPU timings, CUDA code, kernel compilation, correctness evaluation, thermal behavior, clock-state transitions, autocorrelation, or multi-candidate search. The simulated common drift is deliberately large relative to independent jitter to make the effect of pairing measurable. The resulting rates are properties of this model and seed, not estimates of real-world error rates.

The use of a normal critical value with estimated variance is an approximation at the tested sample sizes. The current simulation also does not model optional stopping, adaptive sample sizes, or correction for screening many candidates. A device study should use randomized paired blocks, calibrate the decision threshold from repeated null comparisons, compare against a paired t or bootstrap interval, and assess both false promotion and missed useful improvements.

7. Conclusion

We proposed a fixed-sample, paired confidence-bound rule for deciding whether a candidate kernel should replace a retained reference. In a transparent synthetic experiment with shared drift, pairing reduced timing variability and improved detection of meaningful gains at low false-promotion rates. The result supports testing noise-aware promotion as an addition to self-evolving kernel systems; hardware measurements are required before making performance or deployment claims.

References

[1] Y. Chen et al., “ForgeKernel: Self-Evolving Kernels Catching and Surpassing Experts and Agents,” arXiv:2611.04998v1, 2026. https://arxiv.org/abs/2611.04998